

REAK Platform

Complete Product Requirements & Architecture Specification

September 2026

Contents

1. Executive Summary	4
2. System Architecture	5
2.1 High-Level Shape	5
2.2 Identity & Tenancy Model	5
2.3 Listings & Demands — Mirrored Structures	6
2.4 Core Flows	6
Flow A — Membership & Onboarding	6
Flow B — Listing Lifecycle	6
Flow C — Matching Engine	6
Flow D — Collaboration & Contact Disclosure	7
Flow E — Admin CMS	7
2.5 Security Model Summary	7
2.6 Implementation Stage Order	7
3. Repository & Stack Discovery (First Action, Before Any Code)	9
4. Applications & Routes	10
4.1 Public Website	10
Homepage Sections	10
4.2 Member Portal	10
Dashboard (database-backed, no fake numbers)	10
4.3 Admin Portal	10
Admin CMS Scope	11
5. Design System	12
6. Authentication	13
7. Authorization (RBAC)	14
8. Property Exchange	15
8.1 Discovery	15
8.2 Listing Creation (multi-step)	15
8.3 Data Model	15
8.4 Contact Privacy	15
8.5 Media	15
9. Demands / Client Requirements	16
9.1 Demand Search	16

10. Nepal Location Hierarchy	17
11. Land Area Units	18
12. Matching Engine	19
12.1 Match Explanation	19
13. Collaboration	20
13.1 Workspace	20
13.2 Contact Disclosure	20
14. Notifications	21
15. Feature Flags	22
16. Public Data Security	23
17. Row-Level Security (RLS)	24
18. Storage Security	25
19. Audit Logging	26
20. Security-Sensitive Database Functions	27
21. Database Quality Standards	28
22. Prohibition on Fabricated Business Data	29
23. Responsive Design	30
24. Accessibility	31
25. Performance	32
26. SEO	33
27. Testing Requirements	34
27.1 Unit	34
27.2 Integration	34
27.3 End-to-End Scenarios	34
27.4 Security Testing	34
28. Code Quality	35
29. Environment & Secrets	36
30. Migrations	37
31. Version Control	38
32. Documentation	39
33. Error Handling & Observability	40
34. Production Readiness Checklist	41
35. Governing Principles (Apply Throughout)	42

36. Final Report Format (On Genuine Completion)	43
37. Recommended Execution Approach for Claude Code	44

1. Executive Summary

REAK is a membership-based real estate association platform for Nepal, consisting of three coordinated applications sharing one backend and one database:

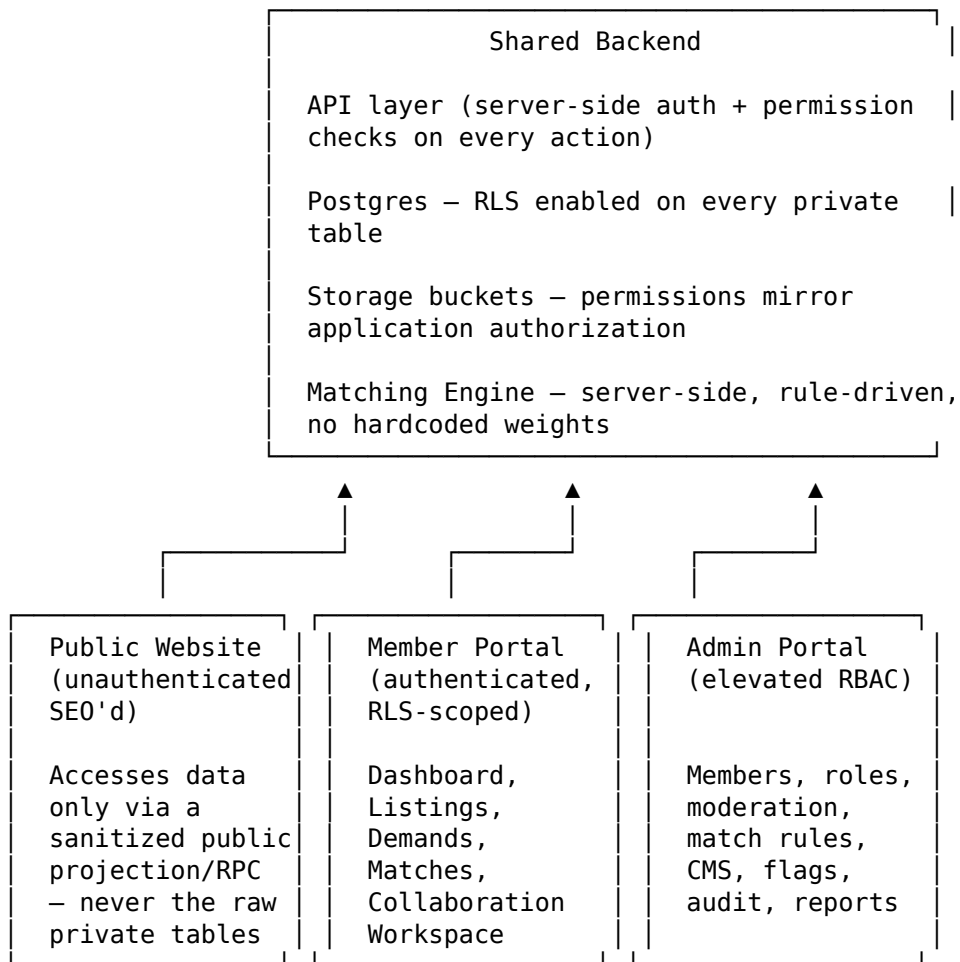
1. **Public Website** — unauthenticated, SEO-optimized, presents the association, its members, news, and (optionally, behind a feature flag) a public property showcase.
2. **Member Portal** — authenticated workspace where member organizations manage property listings, client requirements (“demands”), matches between the two, and collaboration with other members.
3. **Admin Portal** — internal association administration: member management, moderation, CMS, matching-rule configuration, feature flags, audit, and reporting.

The platform’s defining constraints are: **no invented business content, no invented matching weights, contact information physically isolated until explicitly disclosed, and every private table protected by row-level security (RLS), not merely hidden in the UI.**

This document is the complete source of truth for implementation. It supersedes assumptions about technology stack — the actual stack is determined by inspecting the existing repository, per Section 3.

2. System Architecture

2.1 High-Level Shape



Non-negotiable rule: the public site never queries `property_listings` or any private table directly. It goes through a sanitized public projection (view, RPC, or equivalent) that exposes only explicitly approved fields, gated on: public feature enabled AND listing active AND approved AND public visibility AND member active AND not deleted AND not expired. The same principle applies to public member-profile data.

2.2 Identity & Tenancy Model

The platform is multi-tenant. A person (profiles) can belong to a member organization (`member_entities`) through a join table (`entity_users`), and holds roles that are either:

- **System-level** — Super Administrator, Association Administrator. Not tied to any member organization.
- **Organization-level** — Member Administrator, Member Staff. Scoped to one `member_entity`.

Authorization is permission-based (RBAC), not a single `is_admin` boolean. Frontend checks are for UX only; the database (RLS) and server-side permission checks are authoritative. Suspension must revoke access immediately, even for an already-active session.

Core tables: profiles, member_entities, entity_users, invitations, membership_applications, roles, permissions, role_permissions, user/profile role assignments.

2.3 Listings & Demands – Mirrored Structures

Property listings and client demands (requirements) follow the same structural pattern deliberately, since the matching engine treats them as two sides of one relationship:

Listings	Demands
property_listings	demands
listing_media	—
listing_documents (private)	—
listing_amenities	demand_amenities
listing_contacts (isolated, RLS-locked)	demand_contacts (isolated, RLS-locked)
listing_visibility_members	demand_visibility_members
—	demand_property_types
—	demand_locations

Contact isolation is structural, not cosmetic. `listing_contacts` and `demand_contacts` must never be joined into normal read queries — they are separate tables, RLS-locked to the owning organization by default, and only become visible to another organization through an explicit, auditable `collaboration_contact_disclosures` grant. This must be impossible to leak via an incautious `SELECT *`, not merely hidden in the UI layer.

2.4 Core Flows

Flow A — Membership & Onboarding

1. Visitor submits membership_applications via the public site.
2. Admin reviews and approves.
3. Admin creates an invitations record; invitation is emailed.
4. Invitee accepts, sets a password, account activates.
5. entity_users row links the new profile to its member organization.
6. User logs into the Member Portal, scoped to their org's data only.

Flow B – Listing Lifecycle

[illegible]

Moderation (“Pending Review”) only applies if `property_moderation_required` is enabled; otherwise Draft → Approved directly on submit. Visibility to the member network and visibility to the public site are **separate gates** — a listing can be network-visible without being public-visible even when the public feature flag is globally on.

Flow C – Matching Engine

1. A listing or demand is created or updated.
2. Engine checks: is there a **published** match_rule_sets row?
 - **No** → show “Matching is not yet configured by REAK.” Do not generate zero-score or placeholder matches.

- **Yes** → evaluate against the opposite side using `match_rules`, producing matches and per-criterion `match_components` (pass/fail/partial, price delta, area delta, missing data).
3. Member sees the score, the rule-set version used, and a full explanation.
 4. Member can shortlist, dismiss, reopen, request collaboration, or report incorrect data.

No matching weights are hardcoded in application code. All scoring logic lives in admin-configured, versioned `match_rule_sets` / `match_rules` rows — this is what makes results explainable and auditable after the fact.

Flow D — Collaboration & Contact Disclosure

1. Member B sends a `collaboration_requests` (pending) — typically from a match.
2. Member A accepts (or declines/cancels).
3. On acceptance, a `collaborations` workspace is created with participants, messages, files, notes, tasks, and viewing records.
4. **Contact information remains private inside the workspace** until Member A makes an explicit `collaboration_contact_disclosures` grant (tracked: collaboration, data type, granting org/user, timestamp, revocable, policy version).
5. Only after that explicit grant does Member B see contact details. Acceptance of collaboration alone never unlocks contact info.

Flow E — Admin CMS

Content (homepage, about, mission, news, notices, events, resources, legal pages, navigation, SEO settings) moves through Draft → Review → Published → Archived. Published content is what the public site renders; there is no direct “live edit” of public output.

2.5 Security Model Summary

Layer	Enforcement
Frontend role checks	UX convenience only — never trusted for security
API layer	Server-side permission check per action (permission slug vs. caller’s roles)
Database	RLS enabled on every private table — the actual authority
Public data access	Never direct table queries — sanitized RPC/view with explicit field allowlist only
Contacts	Physically separate tables; disclosed only via explicit, revocable, audited grant
SECURITY DEFINER functions	Fixed <code>search_path</code> , strict input validation, minimal privilege, no dynamic SQL
Audit logs	Append-only; no full sensitive-record dumps; no unnecessary PII duplication
Storage	Bucket-level permissions mirror application authorization; private buckets never publicly reachable by path guessing

2.6 Implementation Stage Order

Recommended order (also used to sequence Claude Code sessions — see Section 10):

1. Repository audit + architecture confirmation
2. Design system + public website foundation
3. Database + migrations + RLS + RBAC
4. Authentication + member organizations
5. Member portal
6. Property Exchange
7. Demand/Requirement system
8. Matching engine
9. Collaboration
10. Notifications
11. Admin + CMS
12. Feature flags + reports + audit
13. Security hardening
14. Performance + accessibility + responsive QA
15. Full regression testing
16. Documentation + production readiness

Each stage depends structurally on the one before it — e.g., Matching (8) cannot be validated until both Listings (6) and Demands (7) exist; Collaboration (9) typically originates from a match, so it follows Matching. Skipping ahead produces defects that surface only much later, during security or integration testing.

3. Repository & Stack Discovery (First Action, Before Any Code)

Before writing anything, the implementing agent must determine — by inspection, not assumption:

- Current working directory and existing project structure
- Frontend framework, backend architecture, database, and package manager actually in use
- Existing Supabase (or equivalent) configuration
- Existing routes, components, authentication, migrations, tests
- Environment files and conventions
- Any existing `.claude/` agent/workflow/skill framework and MCP configuration
- Existing documentation

Do not assume Next.js, React Router, Supabase, specific folder names, package manager, or deployment provider until verified. If the repository already implements the approved REAK stack, preserve it. If it differs, evaluate whether migration is genuinely necessary before changing architecture — do not migrate by default.

4. Applications & Routes

4.1 Public Website

/, /about, /leadership, /members, /members/:slug, /membership, /membership/apply, /verify-member, /news, /news/:slug, /notices, /notices/:slug, /events, /events/:slug, /resources, /contact, /properties, /properties/:slug, /login, /forgot-password, /reset-password, /privacy, /terms.

Homepage Sections

1. Header
2. Hero
3. REAK introduction
4. Mission / Vision
5. Association benefits
6. Member Property Exchange explanation
7. How the network works
8. Verified members
9. News
10. Notices
11. Events
12. Membership CTA
13. Public property section (behind feature flag)
14. Contact CTA
15. Footer

Content must be CMS-driven and configurable. **Do not invent official REAK content** (names, numbers, statistics, testimonials, sponsors). Where real content is unavailable, build elegant empty/configuration states rather than filling in placeholder facts.

4.2 Member Portal

/portal/dashboard, /portal/properties, /portal/properties/new, /portal/properties/:id, /portal/properties/:id/edit, /portal/my-properties, /portal/demands, /portal/demands/new, /portal/demands/:id, /portal/demands/:id/edit, /portal/matches, /portal/matches/:id, /portal/collaborations, /portal/collaborations/:id, /portal/members, /portal/members/:id, /portal/saved, /portal/notifications, /portal/profile, /portal/organization, /portal/settings.

All portal routes: authenticated, permission-protected, RLS-protected, noindex.

Dashboard (database-backed, no fake numbers)

Active Properties, Draft Properties, Active Requirements, Potential Matches, Collaboration Requests, Saved Properties, Expiring Items, Recent Properties, Quick Actions.

4.3 Admin Portal

Dashboard, members, users, invitations, roles, permissions, membership applications, properties, demands, matches, match rules, collaborations, property types, amenities, locations, units, currencies, committee, pages, news, notices, events, resources, media, notifications, feature flags, reports, audit logs, settings, security.

Admin CMS Scope

Homepage, about, mission, vision, leadership, members, news, notices, events, resources, membership, contact, footer, navigation, SEO, legal pages. Lifecycle: Draft → Review → Published → Archived.

5. Design System

Establish: typography scale, spacing system, colors, buttons, inputs, cards, dialogs, tables, badges, alerts, navigation, tabs, dropdowns, pagination, loading states, error states, empty states.

Target feel: authoritative, professional, premium, modern, trustworthy, clean, Nepal-relevant, association-focused.

Avoid: generic AI-SaaS aesthetics, excessive gradients, excessive rounded cards, fake luxury imagery, fake statistics, fake testimonials, decorative animation for its own sake.

Use whatever design/component/animation tooling the existing framework already provides (e.g. design-system skills, component libraries, motion libraries) rather than introducing new ones ad hoc.

6. Authentication

Real authentication only — login, logout, password reset, invitation-based account creation, account activation, suspended-account handling, session handling, protected routes.

Never: fake login, frontend-only authentication, hardcoded credentials, insecure bypasses.

7. Authorization (RBAC)

Permission examples: `members.read`, `members.create`, `members.update`, `members.suspend`, `listings.read`, `listings.create`, `listings.update`, `listings.moderate`, `demands.read`, `demands.create`, `matches.read`, `match_rules.manage`, `collaboration.create`, `collaboration.read`, `cms.manage`, `settings.manage`, `audit.read`.

Server-side permission checks are authoritative. Frontend permission checks exist for UX only. Database RLS is the final authority regardless of what the API layer does.

8. Property Exchange

8.1 Discovery

Search, filtering, sorting, pagination, saved filters, saved properties, property cards, property detail, member attribution, visibility rules — all **server-side**. Never download the full dataset to the browser for client-side filtering.

Filters: property type, subtype, purpose, province, district, municipality, ward, locality, price, area, road width, bedrooms, bathrooms, parking, furnishing, facing, amenities, member, status, expiry.

8.2 Listing Creation (multi-step)

Type → Basic Information → Location → Specifications → Price → Amenities → Photos → Documents → Description → Contact/Collaboration → Visibility → Expiry → Review → Submit.

Support: save draft, continue, back, submit, validation, upload progress, error recovery, unsaved-changes protection.

8.3 Data Model

Core fields: UUID, reference code (server-generated), member entity, creator, updater, title, property type, subtype, purpose, location, landmark, coordinates (where appropriate), currency, price, negotiable flag, land area, built-up area, area unit, road access, road width, road type, facing, bedrooms, bathrooms, floors, parking, furnishing, description, internal notes, visibility, moderation status, lifecycle status, approval, expiry, timestamps, archive/delete state.

Related tables: `listing_media`, `listing_documents`, `listing_amenities`, `listing_contacts`, `listing_visibility_members`.

8.4 Contact Privacy

`listing_contacts` must never be returned by normal property queries. RLS default: only the owning organization can access it. Disclosure only via explicit collaboration authorization (Section 12).

8.5 Media

Public listing images (follow listing visibility) are structurally separate from private documents (require signed/private access, upload/delete restricted to authorized users). Do not assume specific document types (citizenship, Lalpurja, PAN, etc.) unless an admin has configured them.

9. Demands / Client Requirements

Purposes: buyer, tenant, investor, other (admin-configurable). A member can register a client's requirement while keeping the client's identity private.

Tables: demands, demand_property_types, demand_locations, demand_amenities, demand_contacts, demand_visibility_members. **Do not model relationships as arrays** — use proper join tables.

9.1 Demand Search

Members may search/filter compatible requirements where visibility permits. Never expose client phone, email, identity, or confidential notes without explicit authorization.

10. Nepal Location Hierarchy

Province → District → Municipality → Ward → Locality, using normalized IDs (not duplicated free-text strings as the primary relationship). Admin can add, edit, disable, and reorder locations.

11. Land Area Units

Support: Ropani, Aana, Paisa, Dam, Bigha, Kattha, Dhur, Square Feet, Square Metres. Automatic conversion stays **disabled** unless explicitly configured and approved — do not invent conversion coefficients.

12. Matching Engine

Bidirectional: Demand → Property and Property → Demand, entirely server-side.

Tables: match_rule_sets, match_rules, matches, match_components, match_actions.

Requirements: explainable, deterministic, configurable, auditable.

No hardcoded weights (e.g. “30% location, 30% budget, 20% area” or any other arbitrary figure). Admin configures rules. If there is no active published rule set, do not generate score-0 matches — show: *“Matching is not yet configured by REAK.”* Only a valid published rule set can generate production matches.

12.1 Match Explanation

Show: score, rule set/version, matching criteria, partial matches, failed criteria, missing data, price difference, area difference, location compatibility, calculation timestamp.

Actions: shortlist, dismiss, reopen, request collaboration, report incorrect data.

13. Collaboration

Tables: `collaboration_requests` (pending / accepted / declined / cancelled), `collaborations` (created on acceptance).

13.1 Workspace

Participants, messages, files, notes, tasks, viewings, activity history — accessible only to authorized participants.

13.2 Contact Disclosure

Acceptance of a collaboration request **never** automatically exposes private contacts. Explicit `collaboration_contact_disclosures` records: collaboration, data type, granting organization, receiving organization, granting user, timestamp, revocation, policy/version. Enforced with RLS.

14. Notifications

In-app notifications for: matches, collaboration requests, collaboration acceptance/decline, messages, listing approval/rejection, expiry, invitations, account events, association notices. Support unread/read state and mark-all-read.

15. Feature Flags

public_properties_enabled, public_member_directory_enabled, open_registration_enabled, membership_application_enabled, property_moderation_required, matching_enabled, collaboration_enabled, deal_tracking_enabled, auto_unit_conversion, sms_notifications, whatsapp_notifications, email_notifications, member_export_enabled. Defaults should be conservative.

16. Public Data Security

Anonymous users must never directly query the private listings (or member) table. Use a sanitized public projection/function/RPC exposing only explicitly approved fields. Public listing access requires: public properties enabled AND active AND approved AND public visibility AND active member AND not deleted AND not expired. Same principle applies to public member profiles.

17. Row-Level Security (RLS)

Mandatory on every private table. Test direct database/API access to confirm users cannot bypass security by calling APIs directly. Suspended users must lose access even with an existing session.

18. Storage Security

Separate storage areas for: CMS public media, member public assets, listing media, private listing documents, membership documents, collaboration files. Storage permissions must mirror application authorization. Private storage paths must never be publicly guessable/reachable.

19. Audit Logging

Audit: login/security changes (where possible), role changes, member changes, listing changes, demand changes, moderation actions, collaboration events, contact disclosure, feature flag changes, configuration changes. Do not blindly store entire sensitive records, and never duplicate private contact information unnecessarily. Audit logs are append-only; normal users cannot modify or delete them.

20. Security-Sensitive Database Functions

For any SECURITY DEFINER function: fixed search_path, explicit permissions, strict parameter validation, no privilege escalation, no arbitrary SQL execution, minimal privileges. Every such function must be reviewed.

21. Database Quality Standards

UUIDs, foreign keys, indexes, unique constraints, check constraints, correct nullability, timestamps, and a defined archive/soft-delete strategy. Reference numbers are generated server-side/database-side, never client-side.

22. Prohibition on Fabricated Business Data

Never invent production members, listings, client requirements, transactions, executive names, phone numbers, emails, addresses, statistics, testimonials, sponsors, or legal claims. Where the UI requires content that doesn't exist yet, use configuration/empty states.

23. Responsive Design

Must work on desktop, laptop, tablet, and mobile — not simply shrunk desktop tables. Use mobile cards, filter drawers, responsive forms, responsive navigation, and touch-friendly actions. Verify actual viewport behavior, not just breakpoint code.

24. Accessibility

Target WCAG 2.2 AA: semantic HTML, keyboard navigation, focus states, labels, error messaging, contrast, accessible dialogs, screen-reader support, reduced-motion support.

25. Performance

Lazy loading, code splitting, pagination, indexed queries, image optimization, debounced search, caching where appropriate, skeleton states, error boundaries. Measure before optimizing — do not optimize blindly.

26. SEO

Public pages: metadata, canonical URLs, Open Graph tags, sitemap, robots.txt, semantic markup. Private routes: noindex.

27. Testing Requirements

27.1 Unit

Validation logic, utilities, matching rules, permission logic.

27.2 Integration

Authentication, database, RLS, API/business logic, storage.

27.3 End-to-End Scenarios

1. Admin creates member
2. Admin invites user
3. User activates account
4. User logs in
5. User creates property
6. Property persists
7. User edits property
8. Another member sees permitted property
9. Private contact remains private
10. User creates demand
11. Demand persists
12. Matching rules are published
13. Match is generated
14. Match explanation appears
15. Collaboration request is sent
16. Collaboration is accepted
17. Collaboration workspace opens
18. Contact remains private
19. Explicit contact disclosure works
20. Suspended user loses access
21. Public feature flag blocks/enables public property visibility
22. Admin CMS changes appear publicly

27.4 Security Testing

Explicitly attempt and confirm failure of: accessing another organization's property; modifying another organization's property; reading private contacts; reading private documents; accessing another organization's demands; modifying roles; granting permissions without authorization; accessing Admin routes without privilege; accessing a collaboration without being a participant; accessing disabled features; bypassing frontend restrictions via direct API calls; accessing data after suspension. All findings must be fixed before completion is declared.

28. Code Quality

Strict TypeScript, clear naming, small focused modules, reusable components, schema validation, typed APIs, proper error handling, logging where appropriate, no dead code, no unnecessary duplication. No TODO placeholders for core features, no fake API responses, no simulated database data, no any abuse, no hardcoded secrets, no commented-out production code.

29. Environment & Secrets

Never hardcode API keys, passwords, Supabase secrets, OAuth secrets, or tokens. Use environment variables; never commit `.env` files with real secrets. Maintain `.env.example` listing variable names only.

30. Migrations

All database changes go through versioned migrations — never manual, undocumented production schema changes. Each migration: deterministic, reviewable, reversible where practical, tested. Do not drop existing data without explicit justification.

31. Version Control

Before meaningful commits: inspect `git status`, review the diff, confirm no secrets are included, confirm unrelated files weren't touched, confirm tests pass. Use meaningful, scoped commits — not large commits mixing unrelated changes. Do not push to a remote unless repository policy explicitly allows it or it is explicitly requested.

32. Documentation

Maintain: README, architecture documentation, database documentation, environment documentation, setup instructions, development instructions, authentication instructions, deployment instructions, security documentation, API documentation (where applicable), testing instructions. Documentation must reflect the actual implementation — never document features that don't exist.

33. Error Handling & Observability

Every important user-facing operation needs: loading state, success state, empty state, validation errors, server-error handling, and retry/recovery where appropriate. Never expose raw database errors to end users; log technical detail appropriately instead.

Log: authentication failures, authorization failures, important business actions, matching events, collaboration events, security events, background job failures. Never log passwords, access tokens, unnecessary private contact data, or secrets.

34. Production Readiness Checklist

Frontend — build succeeds; no TypeScript errors; no console errors; no broken routes; responsive; accessible.

Backend — validation; error handling; authorization; logging.

Database — migrations applied; indexes present; constraints enforced; RLS enabled; functions reviewed.

Security — authentication; RBAC; tenant isolation; PII isolation; storage policies; audit logging.

Testing — unit; integration; E2E; security.

Performance — critical queries, property search, demand search, matching, image delivery all verified.

35. Governing Principles (Apply Throughout)

- Do not stop at a mockup, a database schema alone, a frontend alone, or Phase 1. A feature is complete only when UI → validation → backend → database → RLS → storage → search → detail page → permissions → tests all function together end-to-end.
- No fake data flow: no button → `console.log()`, no button → fake success toast, no API → hardcoded JSON, no dashboard → fake numbers.
- Priority order when conflicts arise: **Security → Correctness → Data integrity → Real functionality → Maintainability → Accessibility → Performance → Visual quality**. Never trade security for convenience, functionality for polish, or maintainability for speed.

36. Final Report Format (On Genuine Completion)

When implementation is genuinely complete, produce a **REAK Production Build Report** covering: project structure; completed features; full route list; database (tables, migrations, functions, indexes, RLS policies); authentication flow; RBAC (system- and org-level roles); security (RLS, tenant isolation, PII protection, public projections, storage, audit); matching implementation and rule configuration; collaboration and contact disclosure; testing results (unit/integration/E2E/security/accessibility, with pass/fail counts); performance findings; accessibility (WCAG) findings; CMS functionality; required environment variable **names only** (never values); production deployment steps; remaining configuration REAK itself must supply (logo, legal name, colors, committee info, contact info, membership policies, approved matching rules, approved area-conversion rules); known limitations (stated honestly); and a final status of exactly **READY FOR PRODUCTION** or **NOT READY FOR PRODUCTION** (with explicit blockers listed if not ready).

37. Recommended Execution Approach for Claude Code

This specification is intentionally large — it describes a full enterprise platform, not a single session’s work. To get a reliable result:

1. Save this document in the repository (e.g. docs/REAK-requirements.md) so it persists as a referenceable source of truth across sessions, rather than re-pasting it each time.
2. Work through Section 2.6’s stage order as **separate, scoped sessions** — each session should name the specific stage, read this document plus the current repository state, and implement only that stage to completion (including its own tests) before moving on.
3. At the start of every session, require a fresh repository inspection (Section 3) rather than trusting assumptions carried over from a previous session’s summary.
4. Do not accept a stage as “done” until its own slice of Section 27 (testing) and Section 17 (RLS) pass — do not defer all testing to the very end.
5. Treat Section 36’s final report as the actual completion gate for the whole project, not a formality — if it cannot honestly state READY FOR PRODUCTION, the project is not finished.